SCIENCE HUB

The Complete Technical & Business Documentation ("The Science Hub Bible")

GST Billing & Inventory Management System
Prepared for engineering, operations, and business stakeholders
Generated 2026-07-24 | Verified directly against the live source code

Table of Contents

> **How this document was built:** every claim in this document was verified by reading the actual source code at `d:\nextApps\science-hub` — not guessed, and not copied blindly from the project's own internal notes (`CLAUDE.md` , `STRUCTURE.md`). Several places where those internal notes turned out to be outdated are called out explicitly in grey "Documentation Discrepancy" boxes throughout, so this Bible reflects the application as it truly runs today.

1. Executive Summary

What is Science Hub?

Science Hub is a web-based business management system built for a company that sells laboratory and scientific supplies (chemicals, glassware, instruments, safety equipment, consumables). In plain terms, it is the digital replacement for the paper ledgers, Excel sheets, and manual invoice books a supplies business would otherwise use to run itself day to day.

It covers the full commercial cycle of such a business:

- Keeping a catalogue of products, their brands, categories, prices, and how much stock is on the shelf.
- Creating GST-compliant sales invoices for customers, automatically calculating the correct tax.
- Recording payments as customers pay, and knowing at a glance who still owes money.
- Handling product returns from customers by issuing credit notes.
- Buying stock from vendors/suppliers, recording purchase bills, and paying those bills.
- Keeping an audit-proof ledger of every single stock movement — every sale, purchase, return, edit, and cancellation.
- Producing management reports — sales, purchases, outstanding balances, GST summaries, and a ready-to-file GST return package.
- Emailing professional PDF invoices directly to customers.
- Controlling who on staff can see or touch what, with a full audit trail of every action anyone takes.
- A "recycle bin" so that accidental deletions are never permanently lost for 30 days.

What business problem does it solve?

A scientific-supplies trading business deals with GST tax rules, stock that must never be oversold, customers who pay in instalments, and a legal requirement to keep clean paperwork for tax filing. Doing this by hand (or in loose spreadsheets) is slow and error-prone: invoice numbers get duplicated, stock counts drift from reality, GST math is done inconsistently, and there is no record of who changed what. Science Hub solves all of this by making every one of these steps a single, auditable, automatically-calculated action inside one system.

Who uses it?

Role	Who they typically are	What they can do
ADMIN	Business owner / senior manager	Everything — full read/write access to all data, user management, business settings, bank details, permanently deleting records, GST filing, and a "factory reset" to wipe all transactional data.
STAFF	Day-to-day billing/accounts clerk	Full read/write on invoices, customers, products, purchases, payments etc. Can be individually granted or denied access to six specific

		"sections" (overview dashboards, reports, and payment histories) by an admin.
MANAGER	Supervisor who needs visibility but not editing rights	Same section-based visibility model as staff, but is **blocked from creating, editing, or deleting anything** anywhere in the system — a strict read-only viewer.

Why was it built?

The codebase and its own internal documentation make clear this is a purpose-built, single-tenant system for one specific business (not a generic multi-company SaaS product) — for example the default seeded business identity is a fictional science-supplies trading company, and the system is deployed once, for one organisation, on Vercel with one Neon database.

Main features (at a glance)

- Invoice creation, editing, and soft-deletion with automatic sequential numbering (`SH-2026-0001` style)
- Automatic GST computation (CGST+SGST for in-state sales, IGST for inter-state sales)
- Product returns issued as numbered credit notes (`CN-2026-0001`)
- Purchase bills from vendors with automatic numbering (`PB-2026-0001`), attachments, and a cancel/un-cancel workflow
- A single, append-only stock movement ledger recording every reason stock ever changed
- Section-based permissions so an admin can fine-tune what a given staff/manager account can view
- Recycle Bin with a 30-day grace period and automatic permanent purge
- Global search across invoices, customers, products, vendors, purchase bills, brands, and categories
- A dedicated GST filing package (Sales Register, Purchase Register, Credit Notes, HSN Summary) exportable as a ZIP for handing to an accountant
- Business branding (name, tagline, logo) that appears on invoices, the login page, and the browser tab
- Full activity log of every meaningful action, by whom and when
- Email delivery of invoice PDFs straight from the app via Gmail

Benefits

- **Accuracy** — GST, discounts, and stock arithmetic are computed once, in one place, the same way every time (never re-typed by hand).
- **Accountability** — every create/edit/delete is attributed to a named user and time-stamped in the activity log.
- **Safety net** — soft-delete and the 30-day recycle bin mean mistakes are recoverable, not catastrophic.
- **Compliance-readiness** — the GST filing package is built directly from the same transactional data used to run the business, so there is one source of truth rather than a reconciliation exercise at tax time.

- **Access control that matches how a small business is actually staffed** — an owner can hand out narrowly-scoped, read-only, or fully-privileged access without needing separate software.

2. High-Level System Overview

At the simplest level, Science Hub works like almost every modern web application: a person uses a web browser, that browser talks to a server over the internet, the server checks a database, and sends the answer back to be displayed. Below is that same idea, specific to how this application is actually built.

1. User opens the Science Hub website in their browser (e.g. on Vercel)

↓

2. The browser downloads the page's React code (built with Next.js)

↓

3. The page checks: is this user logged in? (NextAuth session cookie)

↓

4. The page calls an internal API (e.g. `/api/invoices`) to fetch or save data

↓

5. The API route checks permissions, validates the request, and talks to the database via Prisma

↓

6. The database (PostgreSQL, hosted on Neon) reads or writes the requested rows

↓

7. The API sends a JSON response back to the browser

↓

8. The page updates on screen without a full reload, and its local cache is refreshed

Everything the user sees is rendered "client-side" — every page in this application is explicitly marked `"use client"`, meaning the browser itself does the work of building the visible page after fetching data, rather than the server pre-building the whole page. This is a deliberate architectural choice documented in the project's own instructions, made to avoid a specific bug pattern this version of Next.js has with server-rendered data caching.

The three big layers

Layer	Technology	Responsibility
Presentation (Frontend)	Next.js 16 (App Router) + React 19	Everything the user sees and clicks — forms, tables, dashboards, PDFs on screen.

Application (Backend)	Next.js API Routes (also inside the same project)	Business rules: validation, GST math, stock ledger, permission checks, email sending.
Data (Database)	PostgreSQL on Neon, accessed via Prisma ORM	The permanent record of every customer, product, invoice, bill, and user.

Note: because both the frontend and backend live in the same Next.js project, there is no separate "backend server" to deploy — Vercel builds and hosts both together as one unit.

3. Complete Technology Stack

Every technology below is confirmed present in `package.json` or the source code directly — nothing in this list is assumed.

Core Framework

Technology	What it is (plain English)	Why it's used here
Next.js 16	A framework built on top of React that handles routing (turning URLs into pages), bundling, and server/API code, all in one project.	Lets one project serve both the web pages and the API that powers them, deployed as a single unit on Vercel.
React 19	A JavaScript library for building interactive user interfaces out of reusable "components" (buttons, forms, tables).	Every screen in Science Hub — invoice forms, dashboards, tables — is built from React components.
TypeScript	A version of JavaScript that adds type-checking, catching a class of bugs (e.g. passing a number where text was expected) before the code even runs.	Used throughout the codebase for safer, self-documenting code.

Data & Storage

Technology	What it is	Where/why it's used
PostgreSQL	A mature, industry-standard relational database — data is stored in structured tables with relationships between them (e.g. an invoice belongs to a customer).	The single source of truth for every business record in Science Hub.
Neon	A cloud hosting service that runs PostgreSQL for you, so nobody has to manage a physical database server.	Hosts the production database; connected via a pooled connection string for efficiency under Vercel's serverless model.
Prisma	An "ORM" (Object-Relational Mapper) — a tool that lets developers write database queries as regular code (e.g. `prisma.invoice.findMany()`) instead of raw SQL, and keeps the database structure in sync with a single schema file.	`prisma/schema.prisma` defines all 19 data models; almost every API route uses the generated Prisma client directly to read/write data.
Vercel Blob	A cloud file-storage service (similar in spirit to a private, managed file server) for storing uploaded files.	Stores purchase-bill attachments (scanned bills/photos) and the uploaded business logo.

Authentication & Security

Technology	What it is	Where/why it's used
NextAuth v4	A ready-made authentication library that handles login sessions so the team doesn't have to build login/cookie/token handling from scratch.	Configured in `src/lib/auth.ts` with an email+password ("Credentials") login and JWT (JSON Web Token) sessions lasting 8 hours.
bcryptjs	A one-way password-hashing algorithm — it turns a password into a scrambled string that cannot practically be reversed, so even if the database were stolen, real passwords aren't exposed.	Used to hash every stored user password (bcrypt cost factor 12).
Node's built-in `crypto` module (AES-256-GCM)	A strong, industry-standard symmetric encryption algorithm.	Encrypts two sensitive fields at rest in the database: the Gmail app password and the bank account number, in `src/lib/crypto.ts`.

Communication & Documents

Technology	What it is	Where/why it's used
Nodemailer	A Node.js library for sending email.	Sends invoice PDFs to customers via Gmail SMTP with an "App Password" (a special password Google issues for exactly this purpose).
jsPDF	A JavaScript library that can build a PDF file entirely inside the browser.	Assembles the final downloadable/emailable invoice and credit-note PDF files.
html2canvas	A library that "photographs" a section of a web page and turns it into an image.	Used to capture the on-screen invoice layout so it can be pasted, page by page, into the jsPDF document — this is why invoices always look exactly like what's on screen.
ExcelJS	A library for generating real `.xlsx` Excel files from data.	Powers the "Export to Excel" buttons on Credit Notes, Sales Reports, and Purchase Reports, via a shared `/api/export-xlsx` endpoint.
JSZip	A library for building `.zip` archive files in code.	Bundles the GST filing package (multiple report files) into one downloadable ZIP.

Styling & UX

Technology	What it is	Where/why it's used
CSS Modules	A way of writing CSS so that styles are automatically scoped to one component, preventing accidental style clashes between pages.	Every page/component has its own `*.module.css` file.

CSS Custom Properties (variables)	Reusable named values in CSS (like `--c-bg` for background colour) that can be swapped instantly.	Defined in `src/app/globals.css`; switching the whole app between light/dark mode, or changing the accent colour, is just swapping variable values.

Hosting & Deployment

Technology	What it is	Where/why it's used
Vercel	A cloud hosting platform purpose-built for Next.js applications; automatically builds and deploys the app whenever code is pushed.	Hosts the entire live application.
Environment Variables	Configuration values (secrets, URLs) kept outside the code itself, so the same code can run differently in development vs. production without editing files.	Used for the database connection string, authentication secret, Gmail credentials, and the app's public URL. Full list in Chapter 23.
npm	Node.js's package manager — the tool that installs and tracks every third-party library the project depends on.	Drives every install/build/dev script (see `package.json`).

Not present in this codebase (flagging explicitly rather than guessing, since some outdated internal notes reference them): there is **no Redis**, **no Railway** hosting, and **no AI/Gemini bill-extraction feature** currently in the code — an older internal document (`STRUCTURE.md`) mentions an AI bill-scanning feature and Railway, but no corresponding route or code exists in the current codebase. Rate limiting is implemented with a simple in-memory counter (see Chapter 19), not Redis.

4. Folder Structure

All application code lives under `src/`. Here is what each major folder is for, and why it exists.

```
src/
  app/                          Every page and every API route lives here (Next.js "App Router")
    (dashboard)/                A route "group" — every page a logged-in user sees, sharing one sidebar
      dashboard/                The home KPI dashboard
      sales/                    Customers, Invoices, Payments Received, Credit Notes, Sales Overview
      purchases/                Vendors, Purchase Bills, Payments Made, Purchases Overview
      products/, brands/, categories/     The product catalogue
      reports/                  Sales Reports, Purchase Reports, GST Reports
      admin/                    User management, activity log, section-permissions manager
      bin/                      The Recycle Bin
      settings/                 Business identity, bank details, Gmail, logo, terms & conditions
    api/                        Every backend endpoint — one route.ts file per URL path
    login/, forgot-password/, reset-password/, find-email/      Public, unauthenticated pages
    layout.tsx                  The root page shell — fonts, branding, the anti-flicker theme script
    providers.tsx               Wraps every page in Session / Branding / Theme / Toast context
  components/
    ui/                         Reusable building blocks: Button, Input, Select, Badge, Toast, Paginati
    layout/                     DashboardShell (the real sidebar/topbar/auth-guard), GlobalSearch, Brea
    dialogs/                    ConfirmDialog, PdfCopyDialog
  lib/                          Shared backend/frontend logic (not pages, not UI)
    auth.ts                     Login/session configuration
    apiAuth.ts                  The four permission-check "guards" every API route uses
    prisma.ts                   The single shared database connection
    validation.ts                Every form/field validation rule, shared by client and server
    stockMovement.ts             The stock ledger engine
    invoiceCalc.ts, purchaseBillForm.ts, roundOff.ts     GST/discount/rounding math
    crypto.ts                    Encrypts sensitive settings fields at rest
    activity.ts                  Writes to the audit log
    rateLimit.ts                 Basic abuse-prevention counters
    generateInvoicePdf.ts        Builds the invoice/credit-note PDF
    sections.ts                  Defines the six permission "sections" and what they gate
    businessBranding.tsx         Shares the business name/logo across every page
    gstFiling.ts, gstFilingZip.ts     Builds the GST filing package and its ZIP export
  types/
    next-auth.d.ts               Extends the login session's shape with role/id/sections
prisma/
  schema.prisma                 The single source of truth for the entire database structure
  seed.ts                       Fills a fresh database with realistic demo data for development
```

This structure follows a simple rule: **pages** live under `app/(dashboard)/…/page.tsx`, the **API** they talk to lives at the matching path under `app/api/…/route.ts`, and any logic shared by more than one place is pulled out into `lib/`. There is deliberately no separate "controllers/services/repositories" layering — most API routes talk to Prisma directly rather than going through an extra abstraction layer, which the project's own documentation calls out explicitly as the established convention here (a small, single-tenant application, not a large microservice system).

5. Application Flow

This is the life of a single, typical user session, beginning to end.

Visit the site → redirected to /login if no valid session

↓

Enter email + password → NextAuth verifies against the hashed password in the database

↓

A signed session cookie (JWT) is set in the browser, valid for 8 hours

↓

Redirected to the Dashboard — KPI cards load from /api/reports?type=combined-dashboard

↓

User navigates via the sidebar (only sees items their role/section grants allow)

↓

e.g. Create Invoice → fill form → client validates → submit to /api/invoices (POST)

↓

Server re-validates everything, computes GST, decrements stock, writes the invoice — all inside one database transaction

↓

Invoice detail page can generate a PDF (client-side), download it, print it, or email it to the customer

↓

Every one of the above write actions is recorded in the Activity Log, visible to admins

↓

Deleting anything moves it to the Recycle Bin (soft-delete) rather than erasing it — recoverable for 30 days

Behind every one of those steps, the same repeating pattern is used: the browser calls a `useFetch()` hook pointing at an API route; the API route checks the session, checks the role/permissions, validates the input, talks to Prisma, and returns JSON; the page then updates its own in-memory cache so every open tab/component sees the fresh data without a full page reload.

6. User Roles & Permissions

Science Hub has three roles, stored as a plain text field (`User.role`) — there is no rigid, database-enforced list of allowed roles; the application code is what enforces which three values are meaningful: `admin`, `staff` (the default for new users), and `manager`.

6.1 Admin

ADMIN has unrestricted access to everything. Only an admin can reach: **Settings** (business identity, bank details, Gmail credentials, logo), **Admin** (user management, activity log, section-permissions manager), permanently delete anything from the Recycle Bin, run the **IFSC bank-code lookup**, and trigger a **Factory Reset** (a full wipe of transactional data, covered in Chapter 9). An admin automatically passes every section-permission check — nothing needs to be individually granted to an admin account, and in fact the permissions API explicitly refuses to let anyone set section grants on an admin account.

6.2 Staff (default role)

STAFF can fully create, edit, and delete the core business records — customers, products, invoices, vendors, purchase bills, brands, categories — without restriction. What *is* restricted for staff is visibility into six specific "sections" of the app (dashboards, reports, and payment-history pages) — see 6.4 below. By default a brand-new staff account has none of these six sections enabled and must be granted them by an admin.

6.3 Manager — read-only

MANAGER is functionally identical to staff for the purpose of section-based visibility, but is **hard-blocked from every mutating action** in the system — creating, editing, deleting, recording a payment, cancelling a bill, restoring from the bin — all of it. This is enforced in exactly one shared rule (`requireWriteAccess()` on the server, mirrored by a `useCanWrite()` check on the client) rather than being re-implemented per page, so it is applied consistently everywhere. Managers also cannot see the Recycle Bin link in navigation at all.

6.4 Section-based permissions (an additional, finer-grained layer)

Independently of the three roles above, an admin can grant or revoke access to exactly six "sections" on a per-user basis, for any non-admin user (staff or manager):

Section key	What it controls
`sales_overview`	The Sales Overview dashboard and its KPI widgets on the main Dashboard page
`purchase_overview`	The Purchases Overview dashboard and its KPI widgets on the main Dashboard page

`reports_sales`	The Sales Reports page
`reports_purchases`	The Purchase Reports page
`payments_received`	The Payments Received history page
`payments_made`	The Payments Made history page

Two sections are combined for a seventh, stricter gate: the **GST Reports** page and its underlying API require **both** `reports_sales` *and* `reports_purchases` together (or admin) — a deliberate design decision so that a combined tax-filing report is never handed to someone with only half the relevant visibility.

This is managed from **Admin → Permissions**, a grid where each row is a grantable user and each column is one of the six sections, with a simple on/off toggle per cell. There is no separate "revoke" action — toggling a cell off is the same operation as toggling it on.

> **Documentation discrepancy:** the project's own `CLAUDE.md` / `STRUCTURE.md` notes describe only two roles ("admin" and "staff") and do not mention the manager role or the section-permissions system at all. Direct source inspection confirms all of the above (three roles; a `SectionPermission` database table; a `requireSectionAccess()` guard; an Admin → Permissions page) are real, currently-shipping features that the project's internal notes have not been updated to reflect.

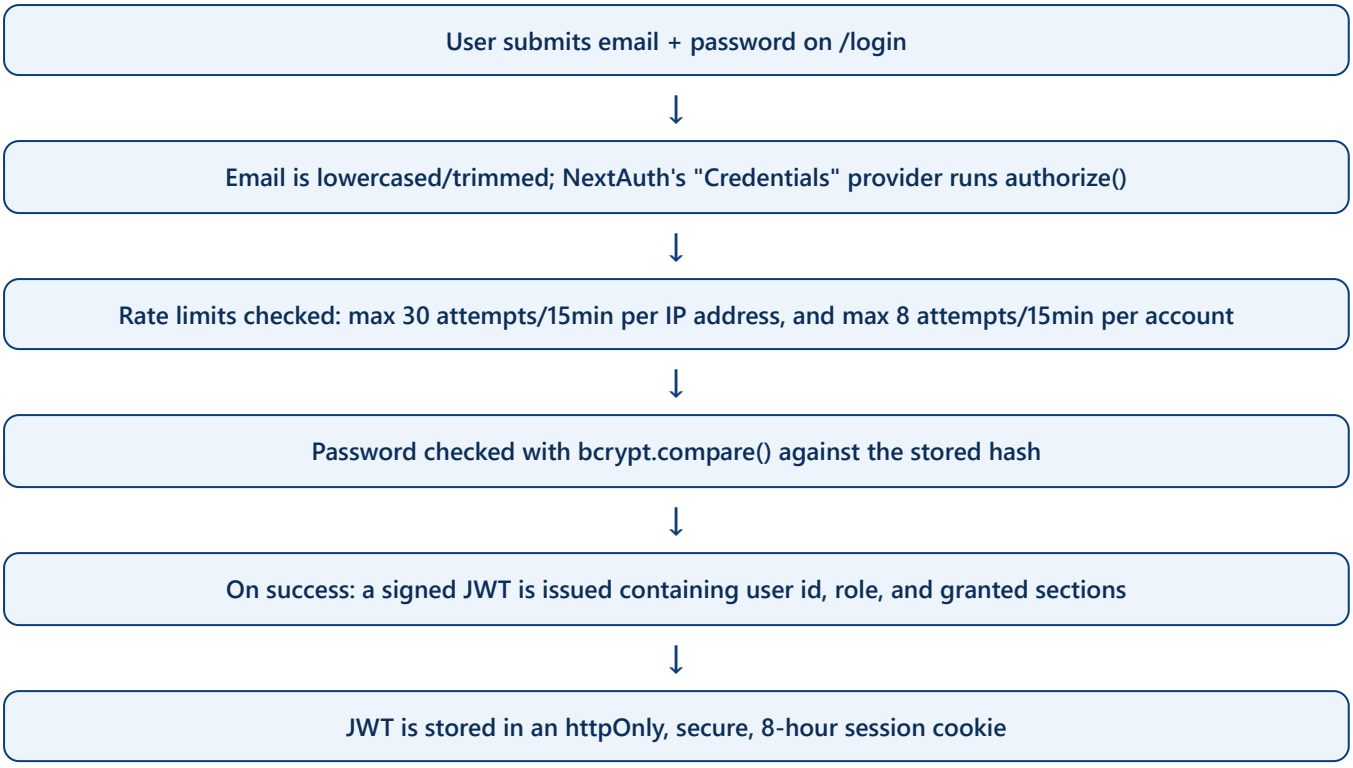
6.5 Permission enforcement summary

Check	Enforced where	Effect
`requireSession()`	Nearly every API route	401 if not logged in at all
`requireAdmin()`	User management, permissions, settings writes, bin permanent-delete, factory reset, stock-ledger clear	403 unless role is admin
`requireWriteAccess()`	Every create/edit/delete endpoint for core business data	403 if role is manager
`requireSectionAccess(section)`	Overview dashboards, reports, payment-history APIs	403 unless admin or the section is individually granted

All of the above are enforced **on the server**. The sidebar and page-level redirects that hide navigation items or bounce a manager away from an edit form are a convenience layer only — the real security boundary is always the API route, never the UI.

7. Authentication & Sessions

7.1 How login works



A subtle but important detail: whether the email doesn't exist *or* the password is wrong, the server always still runs a bcrypt comparison (against a fixed dummy hash if no such user exists) before responding. This means both failure cases take roughly the same amount of time to fail, so an attacker cannot use response-time differences to guess which email addresses are registered — a technique called a "timing attack," and this dummy-hash trick is the standard defence against it.

7.2 What's inside the session

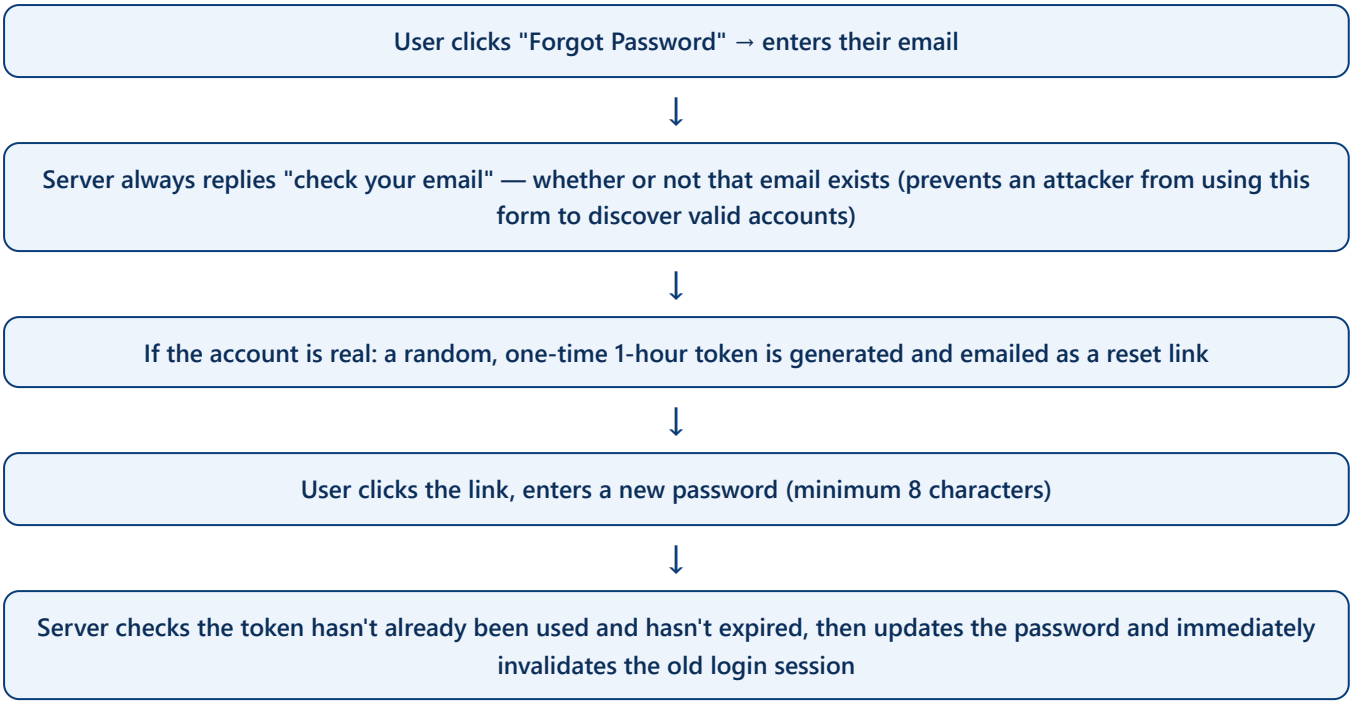
The session available to every page (via `useSession()`) and every API route (via `getServerSession()`) contains: the user's `id`, `name`, `email`, `role`, and `sections` (the array of section keys they've been granted). The role/sections are re-checked against the database periodically (every 30 seconds for sections, every 5 minutes for a full account-validity check) so that an admin revoking someone's access or changing their role takes effect quickly without forcing every user to log out and back in.

7.3 Cookies and JWT — the technical detail, explained simply

A **JWT** (JSON Web Token) is a small, tamper-proof, signed piece of text that encodes who the logged-in user is. Because it's signed with a secret only the server knows, the browser can carry it around in a cookie and the server can trust its contents without needing to look the user up in the database on every single

request. The cookie itself is marked `httpOnly` (JavaScript running in the page can never read it — protects against certain attacks that try to steal login cookies) and `Secure` in production (only ever sent over HTTPS).

7.4 Password reset flow



There is also a separate "Find my email" page: a user can search by their name and get back a masked version of their registered email (e.g. `jo***@example.com`) to jog their memory, without ever exposing the full address or whether a name exists in the system with certainty.

7.5 Password storage

Passwords are never stored in readable form. Every password is hashed with **bcrypt** (a deliberately slow, salted hashing algorithm designed specifically for passwords) at a cost factor of 12 before being saved — even someone with direct database access cannot read anyone's actual password.

7.6 Authorization vs. authentication

Authentication answers "who is this?" (the login/session system above). *Authorization* answers "what is this person allowed to do?" — handled by the four guard functions described in Chapter 6.5. Every single API route in this codebase that touches non-public data calls at least one of these guards before doing anything else; there is no route left unprotected by accident. A historical security audit performed on this codebase (dated 2026-07-03, kept in the repository at `SECURITY_AUDIT_2026-07-03.md`) documents that this was not always the case — earlier in the project's life, roughly twenty API routes checked for a session only to write an audit-log entry but never actually blocked unauthorized access. That audit's fixes were independently

verified against the current code during this documentation effort and are confirmed to be in place today (see Chapter 19 for full detail).

7.7 Is there a "middleware" gatekeeper?

No. Unlike many Next.js applications, this project has no `middleware.ts` file acting as a single front door for every request. Protection is applied per-route instead, using the shared guard functions. This is a valid design, but it does mean a brand-new API route added in the future would be unprotected by default unless a developer remembers to add the guard call themselves — worth keeping in mind as the system grows (see Chapter 28, Future Improvements).

8. Database (Full ER Reference)

The database has **19 tables** (called "models" in Prisma), defined in one file: `prisma/schema.prisma`. There are no database-level enum types in this schema — anything that looks like a fixed set of choices (like an invoice's status) is a plain text field whose allowed values are enforced by the application code, not the database itself.

8.1 Soft-delete vs. permanent records

Has a "Recycle Bin" (deletedAt field)	Never soft-deleted (permanent/append-only)
Customer, Category, Brand, Product, Invoice, Return (credit note), Vendor, PurchaseBill	User, PasswordResetToken, ActivityLog, InvoiceItem, Payment, ReturnItem, BusinessSettings, PurchaseBillItem, PurchasePayment, StockMovement, SectionPermission

The rule of thumb: anything a business person might reasonably want to "undo" deleting (a customer, a product, an invoice) is soft-deleted. Line items, payments, and log entries live and die with their parent record (deleting an invoice cascades to delete its line items and payments in the same instant), and the ledger/log tables are never deleted at all in normal operation.

8.2 Every table, field by field

User — every person who can log in

`id, name, email (unique), password (bcrypt hash), role (default "staff"), tokenVersion (Int, bumped to invalidate sessions on password change), createdAt`

Relations → owns many Invoices, ActivityLogs, PasswordResetTokens, PurchaseBills (as creator), StockMovements (as creator), SectionPermissions

PasswordResetToken

`id, userId → User, token (unique, random 256-bit), expiresAt, usedAt (marks it spent), createdAt`

Deleting a User cascades to delete their reset tokens.

ActivityLog — the audit trail

`id, userId → User, action (e.g. "create_invoice"), details (free text), entityId?, entityType?, createdAt`

Indexed by userId and by createdAt for fast filtering in the Admin activity page.

Customer

`id, name, phone?, email?, address?, city?, state?, pincode?, gstin?, createdAt, updatedAt, deletedAt?`

Has many Invoices.

Category / Brand — catalogue grouping

`id, name (unique), createdAt, updatedAt, deletedAt?`

Each has many Products.

Product

`id, name, description?, sku? (unique), barcode?, hsn?, unit (default "Nos"),`
`price (sale price), purchasePrice?, gstRate (default 18), stock (default 0), minStock (default 5),`
`categoryId?, brandId?, isActive (default true), createdAt, updatedAt, deletedAt?`

Belongs to one Brand and one Category (both optional). Referenced by InvoiceItems, ReturnItems, PurchaseBillItems, and StockMovements.

Invoice — the sales bill

`id, invoiceNumber (unique, "SH-YYYY-0001"), date, dueDate?, customerId → Customer,`
`userId → User (creator), status (unpaid/partial/paid), subtotal, cgst, sgst, igst, roundOff,`
`total, paidAmount, notes?, isInterState, placeOfSupply?, reverseCharge, createdAt, updatedAt, deletedAt?`

Has many InvoiceItems, Payments, and Returns (credit notes).

InvoiceItem

`id, invoiceId (cascade delete), productId, name, hsn, quantity, unit, price,`
`discountPercent, discountAmount, gstRate, gstAmount, total`

Payment — money received against an invoice

`id, invoiceId (cascade), amount, method (default "cash"), reference?, date, notes?`

Return / ReturnItem — a credit note

`Return: id, invoiceId (cascade), creditNoteNumber? (unique, "CN-YYYY-0001"), date, notes?,`
`subtotal, cgst, sgst, igst, roundOff, total, createdAt, deletedAt?`

`ReturnItem: id, returnId (cascade), productId?, name, quantity, price, gstRate, gstAmount, total`

BusinessSettings — one single row, id="singleton"

`name, tagline, logoUrl, email, phone, address, city, state, pincode, gstin, pan,`
`gmailUser, gmailAppPassword (encrypted), bankName, bankAccountName, bankAccountNumber (encrypted),`
`bankIfsc, bankBranch, termsAndConditions, showLogoOnInvoices, updatedAt`

There is exactly one row of this table, ever — the whole application's identity, letterhead, and mail-sending credentials.

Vendor — a supplier the business buys from

`id, name, company?, gstin?, phone?, email?, address?, notes?, isActive (default true),`
`createdAt, updatedAt, deletedAt?`

Has many PurchaseBills.

PurchaseBill

`id, billNumber (unique, "PB-YYYY-0001"), vendorId, billDate, dueDate?, subtotal, taxAmount, discount, roundOff, total, paidAmount, status (unpaid/partial/paid/cancelled), notes?, attachmentUrl?, attachmentName?, category? (free text), createdByUserId, createdAt, updatedAt, deletedAt?`

Has many PurchaseBillItems, PurchasePayments, and StockMovements.

PurchaseBillItem / PurchasePayment

```
PurchaseBillItem: purchaseBillId (cascade), productId?, name, quantity, unit, purchasePrice, discountPercent, discountAmount, gstRate, gstAmount, total

PurchasePayment: purchaseBillId (cascade), amount, method, reference?, date, notes?, createdAt
```

StockMovement — the append-only inventory ledger

```
id, productId? (kept as null forever if the product is later deleted — "SetNull"), productName (a permanent
snapshot of the name at the time, so history survives even product deletion), type (e.g. "sale", "purchase",
"sale_edit_reverse" — see Chapter 11), documentType (invoice/purchase_bill/credit_note/manual), quantity
(signed: positive = stock in, negative = stock out), balanceAfter (the stock level immediately after this
row was written), reference?, notes?, purchaseBillId?, createdByUserId?, createdAt
```

Never updated after creation — a true, permanent audit trail.

SectionPermission — the fine-grained access grants (Chapter 6.4)

`id, userId (cascade), section (one of the six section keys), enabled (default false), createdAt, updatedAt`

One row per (user, section) pair — enforced by a database-level uniqueness constraint.

8.3 Key relationships, plain English

- One **Customer** can have many **Invoices**. One **Invoice** belongs to exactly one Customer.
- One **Invoice** has many **InvoiceItems** (the line items), many **Payments**, and many **Returns** (credit notes issued against it).
- One **Vendor** can have many **PurchaseBills**; the same one-to-many pattern repeats for bill items and payments.
- A **Product** can be referenced by invoice line items, return line items, and purchase-bill line items — but if the product is later deleted, only the **StockMovement** ledger is designed to survive that (its link to the product is simply set to nothing, while the product's *name* is preserved forever as a text snapshot).
- Every meaningful action a user takes can be traced back to them via **ActivityLog.userId**, and every stock change can be traced to both the user who caused it and, if relevant, the exact purchase bill that caused it.

> **Data integrity in practice:** deleting an Invoice automatically and safely deletes its InvoiceItems and Payments (this is enforced by the database itself via "cascade" rules, not just application code) — so it is not possible to end up with orphaned line items pointing at a non-existent invoice.

9. Complete Module Documentation

9.1 Invoices

Purpose: create, edit, and manage GST sales invoices. **Numbering:** `SH-{year}-{0001}`, generated inside a database transaction with automatic retry if two invoices are created at the exact same instant (see Chapter 10 for the full mechanism). **Business rules:** place-of-supply is mandatory on every invoice (required for correct GST); a due date cannot be in the past; the same product cannot appear as two separate line items (quantities must be combined); a fully-paid invoice cannot be edited at all. **Dependencies:** Customers, Products (for stock and GST rate), the stock ledger, and (optionally) the PDF/email system.

9.2 Customers

Purpose: the address book of who invoices are billed to. **Business rule:** a customer with any active (non-binned) invoice cannot be soft-deleted — they must be reassigned or their invoices removed first. A customer can also be created "inline" directly from the invoice-creation screen.

9.3 Products, Brands, Categories

Purpose: the catalogue. Each product has a sale price, an optional purchase price, a GST rate, a current stock count, and a minimum-stock threshold used to flag low stock. **Business rule:** a product used on any invoice line item cannot be soft-deleted; permanently deleting it from the bin additionally checks purchase-bill usage (a slightly stricter check than the everyday soft-delete, described more in Chapter 11).

9.4 Payments (Received)

Purpose: record money received against an invoice. Every time a payment is added or edited, the invoice's paid amount is recalculated from the true sum of all its payments (not just incrementally adjusted), and its status (unpaid/partial/paid) is derived fresh from that sum, so it can never silently drift out of sync.

9.5 Returns / Credit Notes

Purpose: handle a customer returning goods. A return can only be recorded against an invoice that has received at least one payment. It is capped in two independent ways: it cannot refund more than the invoice has actually been paid, and it cannot return more of a product than remains un-returned on that invoice. Every return is numbered like an invoice (`CN-{year}-{0001}`) and restores the returned quantity back into stock.

9.6 Vendors

Purpose: the suppliers the business buys from. Creating a vendor requires phone and address; editing one does not (loosened, since existing vendor records may predate that requirement). A vendor with any active purchase bill cannot be soft-deleted.

9.7 Purchase Bills

Purpose: record a bill from a vendor. Numbered `PB-{year}-{0001}` using the identical mechanism as invoices. Supports an optional file attachment (a photo or PDF of the physical bill), an optional inline first payment at creation time, and a Cancel/Un-cancel workflow distinct from deletion — cancelling reverses the stock the bill added without removing the record, while deleting soft-deletes it entirely (and does not double-reverse stock if it was already cancelled).

9.8 Payments (Made)

Purpose: record money paid to a vendor against a purchase bill. Mirrors the sales-side payment logic (balance re-checked inside a database transaction so two simultaneous payments cannot together overpay a bill).

9.9 Stock & the Stock Movement Ledger

Purpose: the single, permanent, append-only history of every reason a product's stock quantity ever changed. Detailed fully in Chapter 11.

9.10 Reports

Purpose: summarised views for decision-making — this month's revenue/spend, outstanding balances (with an "aging" breakdown of how overdue a purchase bill is), a 12-month GST summary, and management dashboards. Detailed in Chapter 13's API reference.

9.11 GST Filing

Purpose: assembles a ready-to-file GST return package — a Sales Register, a B2B/B2C split, a Credit Notes list, a Purchase Register, and an HSN-code summary — for a chosen month or financial year, downloadable as a ZIP for handing to an accountant. Gated so that only someone (or an admin) with visibility into *both* sales and purchase reports can generate it.

9.12 Settings

Purpose: the business's own identity — name, tagline, logo, GSTIN/PAN, bank details (for printing on invoices), the Gmail account used to send invoice emails, and default Terms & Conditions text. Editable only

by an admin; every section of the settings page saves independently, so editing bank details can never accidentally wipe the Gmail configuration or vice versa.

9.13 Recycle Bin

Purpose: a 30-day safety net for every soft-deletable record type (8 types: invoices, customers, products, brands, categories, vendors, purchase bills, and credit notes/returns). Anything older than 30 days is automatically and permanently purged the next time anyone opens the bin. An admin can also empty the entire bin immediately, or permanently delete one item at a time.

9.14 Admin — Users & Activity Log

Purpose: create/edit/delete staff accounts, assign roles, and browse a searchable, paginated log of every meaningful action taken in the system, by whom, and when.

9.15 Global Search

Purpose: one search box in the topbar that simultaneously searches invoices, customers, products, vendors, purchase bills, brands, and categories (five results per category), with results appearing as the user types.

9.16 Email

Purpose: send an invoice PDF straight to a customer's inbox from inside the invoice detail page, using the business's own configured Gmail account.

9.17 PDF Generation

Purpose: produce a pixel-accurate, print-ready, GST-compliant invoice or credit-note PDF entirely inside the browser — detailed fully in Chapter 17.

10. Invoice Flow — Step by Step

1. User clicks "New Invoice"

↓

2. Selects an existing customer, or fills in a brand-new one inline (name + phone required)

↓

3. Adds line items: product, quantity, price, discount %, GST rate (defaults from the product)

↓

4. Chooses Place of Supply (mandatory) — the app compares it to the business's own state to decide if this is an inter-state sale

↓

5. Client-side validation: quantities > 0, prices ≥ 0, no duplicate products, due date not in the past

↓

6. A warning (not a block) appears if any item would sell more than is currently in stock

↓

7. Submit → POST /api/invoices — the server re-validates everything independently (never trusts the browser)

↓

8. Server computes GST per line (discount is applied first, then GST on what's left), then decides CGST+SGST or IGST for the whole invoice

↓

9. Server generates the next sequential invoice number and, in the very same database transaction, decrements stock for every line item

↓

10. Invoice is saved; the user lands on its detail page

↓

11. Optionally: record a payment, generate/download/print the PDF, or email it directly to the customer

↓

12. Later: the invoice can be edited (reverses old stock, re-applies new stock, blocked once fully paid), have a return/credit note issued against it, or be soft-deleted (which restores its stock and can be undone from the Recycle Bin within 30 days)

Why a "transaction" matters here (explained simply)

A database transaction bundles several steps together as one all-or-nothing unit. If two people tried to create an invoice at the exact same split-second, without a transaction they could both be handed the same invoice number, or one person's stock deduction could be silently lost. Science Hub wraps the whole "generate the next invoice number + create the invoice + adjust stock" sequence inside one **Serializable** transaction (the strictest isolation level a database offers) and automatically retries up to 5 times if the database detects a genuine conflict between two simultaneous attempts — guaranteeing invoice numbers are never duplicated even under real-world concurrent use.

> **Worth knowing:** whether an invoice counts as "inter-state" (and therefore uses IGST instead of CGST+SGST) is decided by the browser, based on comparing the customer's state to the business's own state, and the server currently trusts that flag as sent rather than independently re-deriving it from the place-of-supply field. This is a reasonable trust boundary for a small, single-tenant internal tool, but is worth knowing about if the system is ever exposed to less-trusted clients.

11. Stock Flow & the Movement Ledger

Every single change to a product's stock count — for any reason at all — is written as one row in the `StockMovement` table. This table is never edited or deleted in normal use; it is a permanent, append-only history. The current stock number on the Product itself is really just a "running total" — the ledger is the detailed proof of how it got there.

11.1 Every reason stock can change

Movement type	When it happens	Direction
`sale`	A new invoice is created	Stock decreases
`sale_edit_reverse` / `sale_edit_apply`	An invoice is edited — the old quantities are put back first, then the new quantities are deducted	Reverse (in), then apply (out)
`sale_delete_restore`	An invoice is soft-deleted	Stock increases (restored)
`sale_bin_restore`	A deleted invoice is restored from the Recycle Bin	Stock decreases again (the sale re-applies)
`purchase`	A new purchase bill is created	Stock increases
`purchase_edit_reverse` / `purchase_edit_apply`	A purchase bill's items are edited	Reverse, then apply
`purchase_cancel` / `purchase_uncancel`	A purchase bill is cancelled / un-cancelled	Decrease / increase
`purchase_delete_restore`	A purchase bill is soft-deleted (skipped if it was already cancelled, to avoid double-reversing)	Stock decreases
`purchase_bin_restore`	A deleted purchase bill is restored (skipped if cancelled)	Stock increases
`return`	A credit note is issued against an invoice	Stock increases (goods physically returned)
`return_delete_reverse`	A credit note is soft-deleted	Stock decreases
`return_bin_restore`	A deleted credit note is restored	Stock increases

A "manual" type exists in the code as a reserved category but has no current screen that uses it — likely reserved for a future manual stock-adjustment feature.

> **Recent improvement, visible in the project's own git history:** this specific, named list of movement types replaced an earlier, single generic `"adjustment"` catch-all type — every stock change today

> records precisely *why* it happened, not just *that* it happened.

11.2 How a stock change is actually applied

Whenever multiple line items on one document reference the same product (unusual, but possible), the system first adds up all their quantity changes for that product into one single net number, then applies that one combined change in a single, atomic database update — this avoids a subtle bug where processing each line item separately could let two changes to the same product overwrite each other instead of adding up. The very same database update also returns the product's brand-new stock level directly, which is then written as that row's `balanceAfter` value — meaning the ledger's "running balance" is always taken directly from the database's own result, never separately re-calculated, so it can never drift out of sync even under heavy concurrent use.

11.3 Low stock — three different definitions worth knowing about

Interestingly, the codebase currently computes "is this product low on stock?" three slightly different ways in three different places: the server-side dashboard summary treats zero stock as also being "low stock"; the Products list page's tab counters treat "Out of Stock" and "Low Stock" as two separate, non-overlapping buckets; but that same page's per-row badge on the table goes back to the first, zero-inclusive definition. None of these are wrong exactly, but a business user comparing the dashboard's "low stock" count to the Products page's "Low Stock" tab count may reasonably expect them to match, and today they can differ by exactly the number of products that are completely out of stock.

11.4 Stock & product deletion asymmetry

A product currently used only on purchase-bill line items (never yet sold) can still be soft-deleted from the everyday Products page — that page's block only checks invoice usage. It cannot, however, be *permanently* purged from the Recycle Bin, since that stricter check does look at purchase-bill usage too. This is a two-tier design (soft-delete is more lenient, permanent delete is stricter) rather than a bug, but worth understanding so a "why won't this let me permanently delete it?" question isn't a mystery.

12. GST Logic

12.1 The core formula, per line item

```
gross           = quantity × unit price
discount amount  = gross × (discount % ÷ 100)
taxable amount   = gross − discount amount        ← GST is calculated AFTER discount
GST amount       = taxable amount × (GST rate % ÷ 100)
line total       = taxable amount + GST amount
```

This exact formula is used identically for invoices, purchase bills, and returns/credit notes — confirmed as the same shared logic in all three, not three separately-written (and potentially inconsistent) copies.

12.2 CGST + SGST, or IGST?

India's GST law splits tax two different ways depending on whether a sale crosses a state border:

- **Intra-state sale** (buyer and seller in the same state): the tax is split evenly into **CGST** (Central GST) and **SGST** (State GST) — each exactly half the total GST.
- **Inter-state sale** (buyer and seller in different states): the entire tax amount is charged as one line, **IGST** (Integrated GST) — nothing is split.

In Science Hub, this decision is driven by a single yes/no flag on the invoice ("is this inter-state?"), which the invoice-creation screen sets automatically by comparing the chosen Place of Supply to the business's own registered state.

12.3 Discount

Discounts are entered as a **percentage** per line item (not a flat rupee amount) and — as shown above — are always subtracted before GST is calculated, exactly matching how Indian tax law expects discounts to be treated for GST purposes.

12.4 Reverse Charge

"Reverse Charge" is a GST-law concept where, in certain transactions, the buyer rather than the seller is responsible for paying the GST to the government. Science Hub lets an invoice be flagged as reverse-charge (a simple yes/no checkbox that prints on the invoice as legally required), but this flag is currently for disclosure only — it does not itself change any of the tax arithmetic on the invoice.

12.5 Rounding

After GST is added to the subtotal, the grand total is rounded to the nearest whole rupee (standard commercial rounding — 50 paise and above rounds up), and the small difference this creates (a few paise, positive or negative) is shown as its own explicit "Round Off" line on the invoice, rather than being silently absorbed anywhere.

12.6 Worked example

Item	Qty	Price	Discount	Taxable	GST @18%	Line Total
Beaker 500ml (Borosil)	10	₹120	5%	₹1,140	₹205.20	₹1,345.20

If sold within the same state: CGST ₹102.60 + SGST ₹102.60 = ₹205.20 total GST. If sold to another state: IGST ₹205.20 in one line. Either way the customer pays the same ₹1,345.20 before round-off.

12.7 Credit notes / returns and GST

When a customer returns goods, the credit note re-uses the exact GST rate that was actually charged on the original invoice line for that product (not whatever the product's GST rate might have since changed to) — a small but important correctness detail, since GST rates can legally change over time and a refund must match what was originally charged.

13. API Documentation (Full Reference)

Every endpoint below was confirmed to exist in the current codebase. "Auth" shows the minimum permission guard applied.

13.1 Sales

Method / Path	Auth	Purpose
GET/POST /api/invoices	Session / Write	List invoices (filter by status/customer) · Create an invoice
GET/PUT/DELETE /api/invoices/[id]	Session / Write	View · Full edit (reverses+reapplies stock) · Soft-delete (restores stock)
POST /api/invoices/[id]/payment	Write	Record a payment
PUT /api/invoices/[id]/payment/[paymentId]	Write	Edit an existing payment (no DELETE exists for a single payment)
GET/POST /api/invoices/[id]/returns	Session / Write	List / create a credit note against this invoice
DELETE /api/invoices/[id]/returns/[returnId]	Write	Soft-delete a credit note (reverses its stock effect)
GET/POST /api/customers	Session / Write	List · Create customer
GET/PUT/DELETE /api/customers/[id]	Session / Write	View · Edit · Soft-delete (blocked if active invoices exist)
GET /api/payments	Session	All payments received, flat list
GET /api/credit-notes	Session	All credit notes across every invoice

13.2 Purchases

Method / Path	Auth	Purpose
GET/POST /api/vendors	Session / Write	List · Create vendor
GET/PUT/DELETE /api/vendors/[id]	Session / Write	View · Edit · Soft-delete (blocked if active bills exist)
GET/POST /api/purchase-bills	Session / Write	List · Create purchase bill (auto stock increment)
GET/PUT/DELETE /api/purchase-bills/[id]	Session / Write	View · Edit (blocked on paid/cancelled bill items) · Soft-delete

POST /api/purchase-bills/[id]/payment	Write	Record a payment against a bill
GET /api/purchase-bills/payments	Session	All payments made, flat list
POST/DELETE /api/purchase-bills/upload	Write	Upload/delete a bill attachment (Vercel Blob, magic-byte validated)
GET /api/purchase-reports	Section: reports_purchases	?type=summary | outstanding | category | stock-ledger

13.3 Catalogue

Method / Path	Auth	Purpose
GET/POST /api/products	Session / Write	List (search) · Create product
GET/PUT/DELETE /api/products/[id]	Session / Write	View (incl. last 15 stock movements) · Edit · Soft-delete
GET/POST /api/brands	Session / Write	List · Create brand
PUT/DELETE /api/brands/[id]	Write	Rename · Soft-delete
GET/POST /api/categories	Session / Write	List · Create category
PUT/DELETE /api/categories/[id]	Write	Rename · Soft-delete

13.4 Reports & Filing

Method / Path	Auth	Purpose
GET /api/reports	Session	?type=summary | outstanding | stock | sales-dashboard | purchase-dashboard | combined-dashboard | gst-summary
GET /api/gst-filing	Admin, or reports_sales + reports_purchases both	Builds the GST filing package (JSON, or ?format=zip)
POST /api/export-xlsx	Session	Generic "rows → Excel file" export, used by Credit Notes / Sales / Purchase reports (capped at 20,000 rows)

13.5 System & Admin

Method / Path	Auth	Purpose
GET /api/search	Session	Global search across 7 entity types
GET /api/bin	Write	Auto-purges 30-day-old items, then lists everything still in the bin
POST/DELETE /api/bin/[type]/[id]	Write / Admin	Restore · Permanently delete one item

DELETE /api/bin/empty	Admin	Permanently purge the entire bin at once
GET/PUT /api/settings	Session / Admin	View (non-admins don't see Gmail fields) · Update business settings
GET /api/settings/branding	Public — no login required	Business name/tagline/logo, needed on the login page itself
POST/DELETE /api/settings/logo	Admin	Upload/remove the business logo
GET /api/settings/ifsc-lookup/[code]	Admin	Server-side proxy to look up a bank branch from its IFSC code
GET/POST /api/admin/users	Admin	List · Create staff/admin/manager accounts
GET/PUT/DELETE /api/admin/users/[id]	Admin (self-edit allowed for own profile)	Manage one user; blocks deleting yourself or the last admin
GET/PUT /api/admin/profile	Session	View/update your own profile and password
GET/DELETE /api/admin/activity	Admin	Browse the activity log · Clear the entire log
DELETE /api/admin/activity/[id]	Admin	Delete one log entry
GET/POST /api/admin/permissions	Admin	List/grant section permissions for non-admin users
POST /api/admin/factory-reset	Admin	Wipe all transactional data (see Chapter 9.11 / 28)
DELETE /api/stock-movements?type=stock-ledger	Admin	Bulk-clear the stock ledger's history rows (does not change current stock levels)
POST /api/send-invoice	Write, rate-limited	Email an invoice PDF via Gmail
POST /api/setup	None (disabled once any user exists, or in production)	Create the very first admin account and seed demo data
POST /api/auth/find-email	Rate-limited, no login	Search by name, return a masked email
POST /api/auth/forgot-password	Rate-limited, no login	Start a password reset
POST /api/auth/reset-password	Rate-limited, no login	Complete a password reset
* /api/auth/[...nextauth]	—	NextAuth's own internal login/session handler

14. Frontend Architecture

14.1 Pages & layouts

Every authenticated page lives under the `(dashboard)` route group and is wrapped by one shared component, `DashboardShell`, which provides the sidebar, topbar, global search box, and the client-side authentication gate (redirecting to `/login` the moment a session becomes invalid). Public pages (login, forgot/reset password, find-email) sit outside this shell entirely.

14.2 State management — deliberately simple

There is no heavyweight global state library (no Redux, no Zustand). Instead, server data is managed by one small, custom hook, `useFetch(url)`, backed by a single shared in-memory cache: the first component to ask for a URL fetches it and stores the result; every other component asking for that same URL, anywhere in the app, instantly gets the same cached value and is notified the moment it changes. After any create/edit/delete, the calling page explicitly tells the cache to refresh (`mutate()`) or discards the stale entry (`bustCache()`) — there is no automatic timed expiry; a cached value lives until something explicitly invalidates it.

14.3 Forms & validation

Every input field's validation rule (required, email format, GSTIN format, PAN format, IFSC format, phone number, positive number, etc.) is defined once in `src/lib/validation.ts` and reused on both sides: the browser checks it immediately for a responsive experience, and the very same rule set of checks is re-run on the server before anything is saved — so the server is never simply trusting what the browser already checked.

14.4 Shared UI components

Component	Purpose
Button	One consistent button across the whole app — primary/secondary/danger styles, loading spinner built in
Input / Select / Textarea / FormField	Consistent form fields with label, required-marker, and inline error message
Badge / StatusBadge	Colour-coded pill labels — e.g. green "Paid", amber "Partial", red "Unpaid"
Toast	Temporary pop-up notifications for success/error/warning messages
ConfirmDialog	"Are you sure?" modal used before every destructive action

PdfCopyDialog	Lets the user choose "Original" and/or "Duplicate" copy before generating an invoice PDF
Pagination	Standard page-by-page browsing for long lists, with a "show all" toggle
Skeleton	Grey placeholder blocks shown while data is loading, so the page never flashes empty
GlobalSearch	The topbar search box, debounced (waits 250ms after typing stops) and cancels any still-in-flight previous search

14.5 Theming

Light/dark mode and a customisable accent colour are both controlled purely through CSS variables and a small script that runs before the page even paints — this specifically prevents the common "flash of the wrong theme" bug seen on many websites when the page briefly shows light mode before switching to a saved dark-mode preference.

> **Documentation discrepancy:** the project's own internal notes list "theme flicker on initial load" as a known, unfixed issue. On inspection of the current source code, a proper pre-paint fix (an inline script plus `suppressHydrationWarning`) is actually already present in `src/app/layout.tsx` — the internal notes appear to simply be out of date on this point.

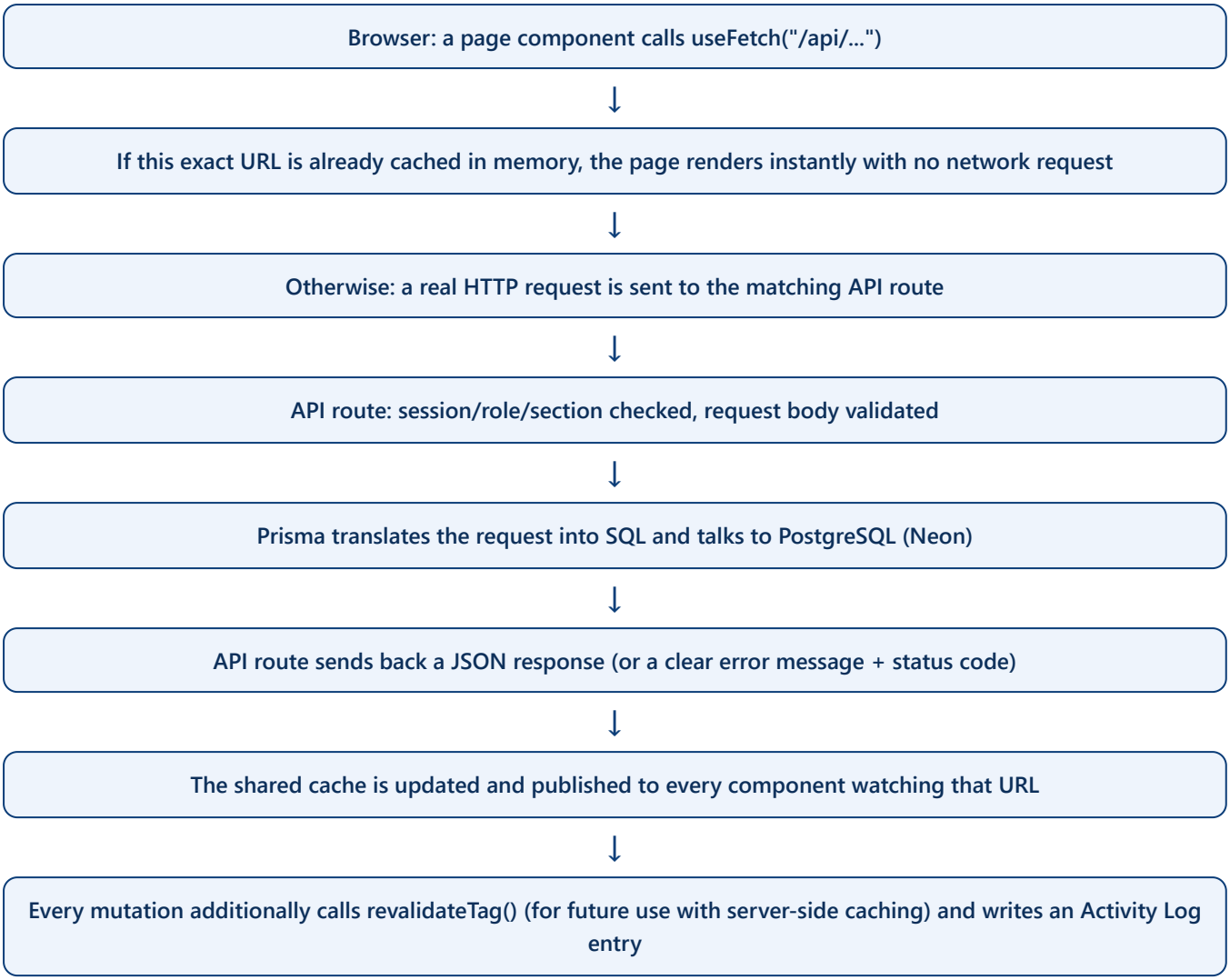
15. Backend Architecture

There is no separate "controller / service / repository" layering in this codebase — it is intentionally kept simple. Almost every API route file directly: (1) checks permissions, (2) validates input, (3) talks to Prisma, (4) logs the action, (5) tells the cache to refresh. A small number of shared helper modules exist for logic that would otherwise be duplicated:

Helper module	Responsibility
`apiAuth.ts`	The four permission-guard functions (Chapter 6.5)
`invoiceCalc.ts`, `purchaseBillForm.ts`, `roundOff.ts`	Shared GST/discount/rounding math, so invoices and purchase bills can never compute tax two different ways
`stockMovement.ts`	The single place every stock change must go through (Chapter 11)
`invoiceReturns.ts`	Prevents editing an invoice's quantities below what's already been returned against it
`blobStorage.ts`	File-upload URL allowlisting and best-effort cleanup for attachments/logos
`activity.ts`	Writes to the audit log; deliberately can never throw an error that would break the real operation
`rateLimit.ts`	Basic abuse-prevention counters for login, password reset, and email sending
`crypto.ts`	Encrypts/decrypts the two sensitive settings fields
`gstFiling.ts` / `gstFilingZip.ts`	Builds the GST filing report and its downloadable ZIP

File uploads (purchase-bill attachments, the business logo) go to Vercel Blob storage. Every upload is checked not just by its declared file type but by inspecting the actual first bytes of the file ("magic bytes") — this stops someone renaming a disguised file to make it look like an allowed type. Every stored file URL is also checked against a strict allow-list before being trusted, deleted, or rendered as a link, closing off a class of attack where a malicious URL could otherwise be smuggled into the database.

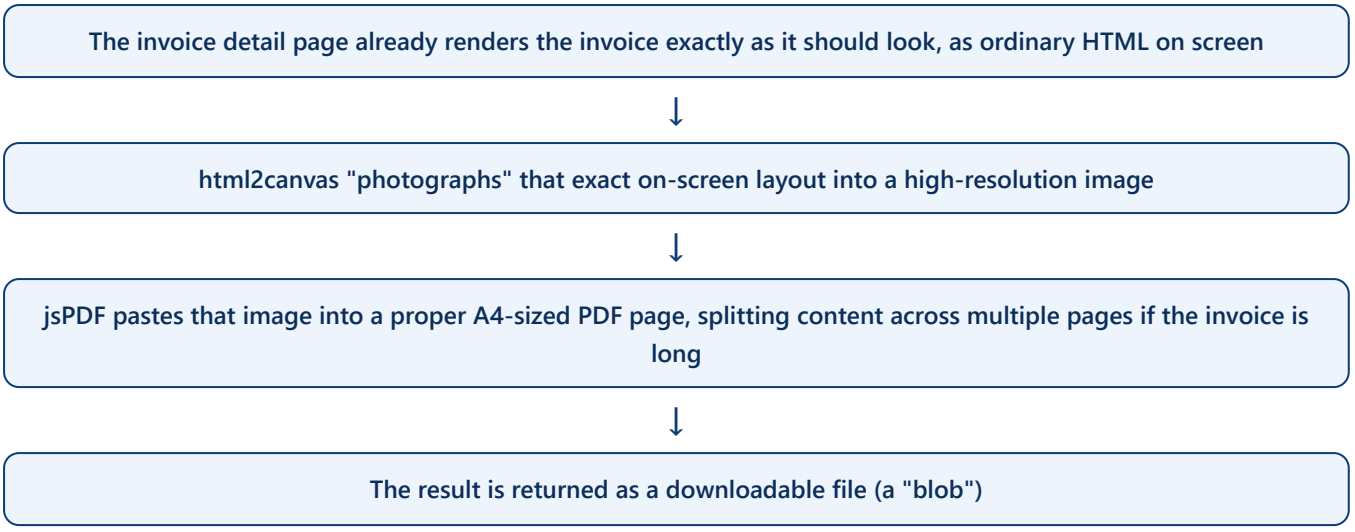
16. Data Flow



Because every page is client-rendered, there is no separate server-side data cache actually in effect today — freshness comes entirely from the client-side cache above plus each page explicitly refreshing after a write. The `revalidateTag()` calls sprinkled through the mutation routes are kept for consistency and in case server-side caching is added in the future, but nothing currently reads from that server cache layer.

17. PDF Generation

Invoice and credit-note PDFs are built entirely **inside the customer's own browser** — nothing is rendered on a server. The technique, in plain terms:



Because printing, downloading, and emailing all funnel through this exact same image-capture pipeline, the invoice a customer receives by email is guaranteed to look byte-for-byte identical to what was printed or downloaded — there is no separate, possibly-inconsistent "print stylesheet" being maintained on the side.

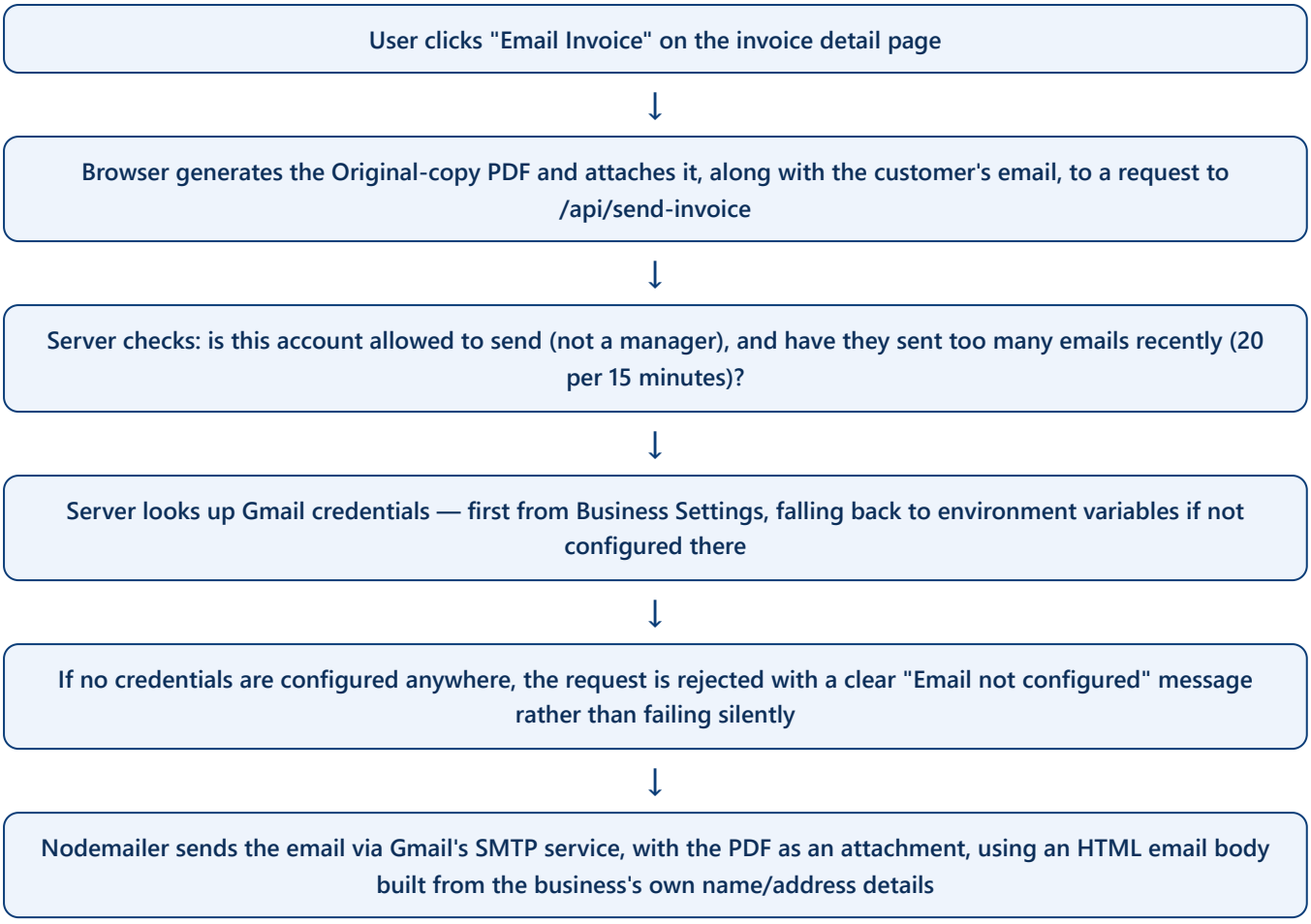
Original vs. Duplicate copies

Indian invoicing convention often calls for multiple labelled copies of the same invoice — an "Original" for the buyer and a "Duplicate" retained by the seller. The **PdfCopyDialog** lets the user choose either or both; if a Duplicate copy is included, a signature block for the receiver is automatically shown only on that copy (since it is the seller's proof-of-delivery record). When an invoice is emailed to a customer, only the Original copy is ever attached — the Duplicate is deliberately reserved for the business's own records.

Handling long invoices

The page-splitting logic is deliberately careful: it never cuts a table row in half across a page break, the invoice footer/terms/bank-details block is always pinned to the very bottom of whichever page it lands on (never floating awkwardly), and once an invoice has 18 or more line items, the system forces everything from the 18th item onward onto a fresh page — even if a little more would technically still fit — so the layout always stays clean and readable.

18. Email System



Every piece of text pulled into the email's HTML (customer name, invoice number, business name/address) is "escaped" before being inserted — meaning special HTML characters are neutralised so that no customer or business data, however it was typed in, can accidentally break the email's formatting or inject unwanted HTML/scripts into the message.

Credential source: the business's own Gmail address and app password, entered once in Settings and encrypted before being stored, are used first; two environment variables (`GMAIL_USER` / `GMAIL_APP_PASSWORD`) exist purely as a fallback for environments where Settings hasn't been configured yet.

19. Security

19.1 Authentication security

- Passwords hashed with bcrypt (cost 12) — never stored in readable form.
- A timing-safe "dummy hash" comparison prevents attackers from telling apart "wrong password" from "no such account" by measuring response speed.
- Dual rate-limiting on login: per-account (8 attempts/15 min) and per-IP-address (30 attempts/15 min).
- Sessions are httpOnly, Secure (production), SameSite-Lax cookies containing a signed JWT, valid for 8 hours.
- Changing or resetting a password bumps a per-user "token version" counter, which invalidates any already-issued session tokens for that account within a few minutes.

19.2 Authorization

All meaningful checks happen server-side via the four guard functions (Chapter 6.5). A prior, documented security audit (kept in the repository, dated 2026-07-03) found that roughly twenty API routes historically checked a session only to write to the activity log, but never actually blocked an unauthorized caller from proceeding. During this documentation effort, the fixes described in that audit were independently spot-checked against the live code and found to match — including the login timing fix, the password-reset enumeration fix, and closing a privilege-escalation path where a client-supplied session update could previously have let a user grant themselves admin rights (the code now always re-derives role/name/email from the database, never from what the client sends).

19.3 Input validation / injection protection

- All database access goes through Prisma, which parameterizes every query — there is no hand-built SQL string concatenation anywhere in the reviewed code, closing off SQL-injection as a risk in normal use.
- Every form field has a defined, reused validation rule (Chapter 14.3), enforced identically on both browser and server.
- Email HTML bodies escape all user-supplied text before insertion (Chapter 18), preventing HTML/script injection into outgoing mail.
- File uploads are checked against their real byte signature, not just their claimed file type, and every stored file URL is checked against a strict allow-list before being trusted or deleted.

19.4 Secrets at rest

Two especially sensitive fields — the Gmail app password and the bank account number — are encrypted (AES-256-GCM) before being written to the database, rather than stored as plain text. Their encryption key is derived from the same secret used to sign login sessions (`NEXTAUTH_SECRET`) rather than a wholly

separate encryption key. This is a reasonable, working protection, but it does mean rotating the login-signing secret and rotating the encryption key are not independent operations today — worth knowing if either is ever rotated in production (see Chapter 28).

19.5 First-time setup safety

The one-time `/api/setup` endpoint (which creates the very first admin account) is completely disabled once the application is running in a production environment, and separately refuses to run at all if the database already has even one user in it — so it can never be used to wipe or reseed a live, in-use system.

19.6 Rate limiting — an honest limitation

Rate limiting (on login, password reset, and email sending) is implemented as a simple in-memory counter, not a shared, persistent store like Redis. This is fine for a single-server deployment, but if the application were ever scaled to run across multiple server instances simultaneously, each instance would keep its own separate counter — meaning the effective limit would be higher than the stated number, and a restart or redeploy resets all counters to zero. This is documented as a known, accepted trade-off in the code itself, not an oversight.

19.7 No default-deny middleware

As noted in Chapter 7.7, there is no single "gatekeeper" file that every request must pass through by default — each route protects itself. This works today because every existing route was verified to include the correct guard, but it does place responsibility on future developers to remember to add one to any brand-new route (see Chapter 28).

20. Error Handling

Frontend

Two distinct patterns are used consistently: a failure to even *load* a page's data shows a small error banner with a retry option; a failure that happens *while saving* something (a validation problem, a conflict, a server error) shows a temporary "toast" notification instead, so the user never has to scroll up to discover what went wrong on a long form.

Backend

Every API route wraps its logic in error handling that distinguishes between different failure types and returns an appropriate, specific HTTP status code and message rather than one generic failure for everything:

Situation	Status
Not logged in	401 Unauthorized
Logged in, but the wrong role/permission for this action	403 Forbidden
The record being requested/edited doesn't exist	404 Not Found
The request itself is invalid (missing field, bad format, business-rule violation)	400 Bad Request
Someone else updated this record since it was loaded on screen (optimistic-concurrency conflict)	409 Conflict
An email attachment is too large	413 Payload Too Large
Too many attempts in a short time (login, password reset, sending email)	429 Too Many Requests
A required external service (e.g. Gmail) isn't configured	503 Service Unavailable
Anything unexpected	500 Internal Server Error, with details only in the server log, never shown to the user

Optimistic-concurrency conflicts, explained

Several edit screens (invoices, purchase bills, customers, vendors, products, settings) send back the record's last-known "updated at" timestamp along with any edit. If someone else has changed that same record in the meantime, the save is rejected with a clear "this was changed by someone else" message rather than silently overwriting their change — protecting against two people editing the same record at the same time without realizing it.

Database

Prisma's own error codes are checked and translated into friendly messages where it matters — for example, trying to reuse a product SKU that's already taken returns a clear "SKU already in use" message rather than a raw database error.

Activity logging never breaks the real action

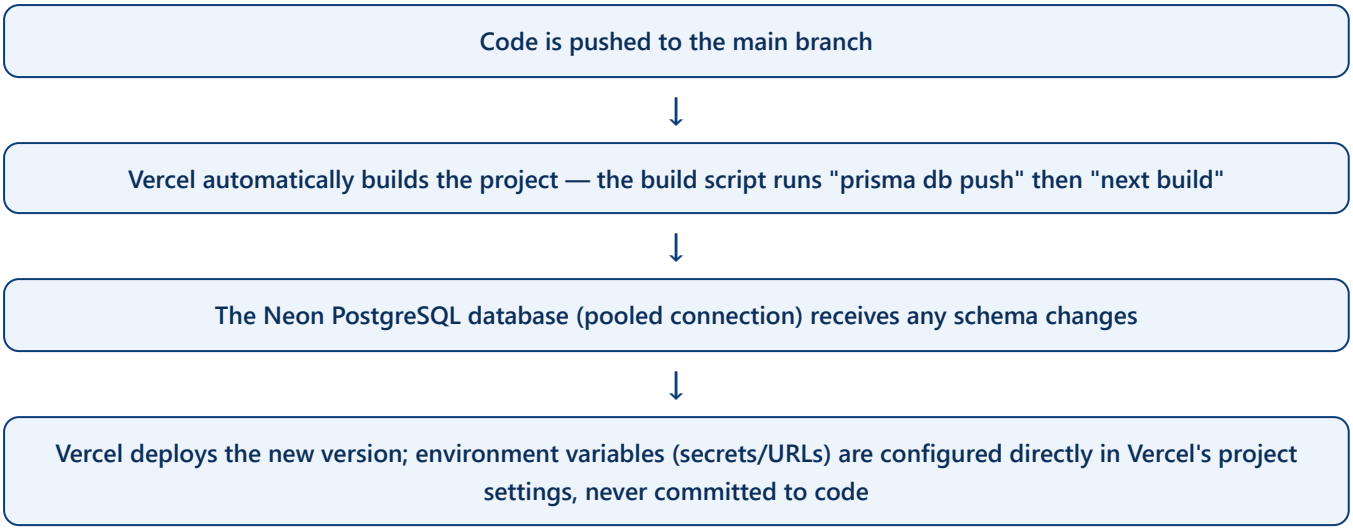
Writing an audit-log entry is deliberately wrapped so that it can never itself cause the real business action (creating an invoice, deleting a customer, etc.) to fail — if logging fails for any reason, it fails silently in the background rather than blocking the user's actual work.

21. Deployment

Development

```
npm install                → also runs "postinstall": prisma generate (builds the database client)
npm run dev                → starts the local development server
npx prisma migrate dev     → applies a schema change locally
npx tsx prisma/seed.ts     → fills a fresh local database with realistic demo data
```

Production



The `postinstall` script (`prisma generate`) is required for Vercel's build to succeed — removing it is explicitly called out in the project's own conventions as something that must never be done.

A note on Windows local development

On Windows, the generated Prisma database client is a locked file while the development server is running — so after any schema change, the development server must be stopped, `npx prisma generate` run, and only then the server restarted, or the generate step will fail.

22. Third-Party Services

Service	Used for	Notes
Vercel	Hosting the entire application	Auto-deploys from the main branch
Neon	Managed PostgreSQL database hosting	Pooled connection string used in production for efficiency
Vercel Blob	File storage for purchase-bill attachments and the business logo	Requires a `BLOB_READ_WRITE_TOKEN`; without it, attachment/logo uploads fail gracefully but nothing else in the app is affected
Gmail SMTP	Sending invoice PDF emails and password-reset emails	Uses a Gmail "App Password," not a full account password or OAuth
Razorpay's public IFSC directory	Looking up a bank branch name/address from an IFSC code when entering bank details	Called only server-side (admin-only), never directly from the browser

23. Environment Variables

Variable	Required?	Purpose
`DATABASE_URL`	Yes	The connection string to the Neon PostgreSQL database (pooled, with a connection-limit of 1, in production)
`NEXTAUTH_SECRET`	Yes	Signs every login session token, and is also the basis for the encryption key used to protect the Gmail app password and bank account number at rest. Must be at least 32 random characters.
`NEXTAUTH_URL`	Production only	The full public URL of the deployed site — used to build password-reset links correctly
`GMAIL_USER`	Optional	Fallback Gmail sending address, used only if Business Settings hasn't configured one
`GMAIL_APP_PASSWORD`	Optional	Fallback Gmail app password, paired with the above
`BLOB_READ_WRITE_TOKEN`	For attachments/logo	Authorizes uploads to Vercel Blob storage; auto-provided on Vercel

24. Project Dependencies

Package	Why it's in this project
next, react, react-dom	The core framework and UI library
@prisma/client, prisma	Database access and schema management
next-auth, @auth/prisma-adapter	Login/session handling
bcryptjs	Password hashing
nodemailer	Sending email
jspdf, html2canvas	Building invoice/credit-note PDFs in the browser
exceljs	Generating downloadable Excel reports
jszip	Bundling the GST filing package into a ZIP
@vercel/blob	File storage for attachments and the logo
dotenv	Loading environment variables during local scripts (e.g. the seed script)
tailwindcss (dev only, via @tailwindcss/postcss)	Present in the toolchain, though the app's actual styling approach is CSS Modules + custom properties rather than Tailwind utility classes
typescript, eslint, eslint-config-next	Type-checking and code-quality tooling
playwright (dev only)	Browser automation/testing tooling available in the project

25. Business Workflow — A Day in the Life

Morning: Owner checks the Dashboard — revenue this month, outstanding balances, overdue invoices, low-stock alerts, at a glance

↓

A shipment of chemicals arrives from a vendor → a Purchase Bill is created, stock automatically increases

↓

A customer calls in an order → Staff creates a new Invoice, GST calculated automatically, stock automatically decreases

↓

The invoice is emailed directly to the customer, or printed for a walk-in customer

↓

The customer pays a few days later (by cheque/UPI/bank transfer) → the payment is recorded against the invoice

↓

A different customer returns a damaged item → a Credit Note is issued, stock is restored

↓

End of month: the owner pays the vendor's bill → payment is recorded, the vendor's outstanding balance drops

↓

Owner reviews Sales/Purchase Reports and, at tax time, generates the GST Filing package to send to the accountant

↓

Occasionally: something is deleted by mistake — it sits safely in the Recycle Bin for 30 days, fully recoverable

26. Non-Technical Explanation

For a shop owner

Think of Science Hub as a smart notebook that never makes an arithmetic mistake, never runs out of pages, and remembers who wrote what and when. Instead of writing an invoice by hand and doing GST maths on a calculator, you type in what was sold, and the system works out the tax, gives the bill its next number automatically, and quietly keeps count of how much stock is left on your shelf — all at the same moment.

For a business owner thinking about risk

Nothing is ever truly deleted by accident — anything removed goes into a "Recycle Bin" for 30 days before it's gone for good, and every single action anyone takes is written down in an activity log with their name and the time. You can also decide exactly what each staff member is allowed to see and do — someone can be given permission to just look at reports without being able to change anything at all.

For a school student, using an analogy

Imagine a library. The **database** is the shelves where every book (piece of information) is actually kept. The **website** you see in your browser is like the front desk — you never touch the shelves yourself. When you ask the front desk for a book (say, "show me this month's invoices"), a librarian (the **API**) goes and fetches exactly that from the shelves and brings it back to you, checking your library card (your **login**) first to make sure you're allowed to see it.

For someone who has never written a line of code

Every button you click in this application quietly sends a small, structured message to a computer somewhere else (the "server"), which checks who you are, does some maths or looks something up in an organized filing system (the "database"), and sends back the answer — which is then drawn on your screen as the numbers and tables you see. Nothing you see is invented on the spot; it always comes from a real, permanent record kept in that filing system.

27. Developer Guide

27.1 First-time setup

```
1. npm install                       (also runs "prisma generate" automatically)
2. Create a .env file with at least DATABASE_URL and NEXTAUTH_SECRET
3. npx prisma migrate dev            (creates the database tables)
4. npx tsx prisma/seed.ts            (fills the database with realistic demo data, prints an admin logi
5. npm run dev                       (starts the app at http://localhost:3000)
```

27.2 First admin account in a fresh production deployment

Send a one-time request to `POST /api/setup` with a name/email/password. It only works while the database has zero users, and is completely disabled once the app is running with `NODE_ENV=production` and even one user already exists — so it can safely be left in the codebase without being a lingering security risk.

27.3 Making a schema change

```
1. Edit prisma/schema.prisma
2. Stop the local dev server (Windows locks the generated client file while it runs)
3. npx prisma migrate dev --name describe-your-change
4. npx prisma generate
5. Restart the dev server
```

27.4 Adding a new page

1. Create `src/app/(dashboard)/<section>/<page>/page.tsx`, starting with `"use client"`.
2. Fetch data with `useFetch("/api/<resource>")`; call `mutate()` after any write.
3. Add an entry to `NAV_GROUPS` in `src/components/layout/DashboardShell.tsx` if it should appear in the sidebar.

27.5 Adding a new API route

1. Create `src/app/api/<resource>/route.ts`.
2. Call the appropriate guard from `src/lib/apiAuth.ts` first, before touching any data.
3. Validate the request body — reuse an existing `rules.*` validator from `src/lib/validation.ts` where possible.
4. Call `logActivity(...)` for any create/edit/delete.

5. Call `revalidateTag(tag, {expire: 0})` after any write (always the two-argument form).

27.6 Rules that must never be broken (confirmed still true in the current code)

- Never change the invoice number format `SH-YYYY-0001` — it is legally printed on issued invoices.
- Never remove the `postinstall` script from `package.json` — Vercel's build depends on it to generate the Prisma client.
- Never mutate a product's stock outside of `batchAdjustStock()` — the ledger must always stay the true, complete history of every stock change.
- Never accept or delete a file URL for an attachment or logo without passing it through the allow-list check first.
- Always use the two-argument form of `revalidateTag(tag, {expire:0})` — the single-argument form is deprecated in this version of Next.js.

> **Two internal documents are out of date** and should be treated with caution by any developer relying on them: `STRUCTURE.md` (mentions an AI bill-scanning feature and Railway hosting that do not exist in the current code, and incorrectly says `NAV_GROUPS` lives in `layout.tsx` rather than `DashboardShell.tsx`) and, to a lesser extent, `CLAUDE.md` (does not yet mention the manager role, section-based permissions, credit notes, GST filing, business branding, or factory reset — all of which are real, working features in the current codebase). This Bible was written by reading the actual code, not these notes, specifically to avoid inheriting their inaccuracies.

28. Future Improvements

These are observations made directly from reading the current architecture — not requests, just honest opportunities worth the team's attention, roughly in priority order.

Security & reliability

- Add a `middleware.ts` default-deny layer so a newly-added API route is protected automatically, rather than relying on every future developer remembering to add a guard by hand.
- Move rate limiting to a shared, persistent store (e.g. Redis/Upstash) so limits hold correctly if the app is ever scaled across multiple server instances, and survive a redeploy.
- Use a separate, dedicated encryption key for secrets-at-rest (Gmail app password, bank account number) instead of deriving it from the same secret that signs login sessions — so the two can be rotated independently.
- Add a database transaction to the "edit an existing payment" endpoint, matching the safety already present on "create a new payment," so two concurrent edits to the same payment can't race each other.

Data consistency

- Unify the three slightly different "is this product low on stock?" definitions found across the dashboard summary, the products-list tabs, and the products-list per-row badge, into one single, shared calculation.
- Make the Product/Brand/Category soft-delete check also consider purchase-bill usage (today it only checks invoice usage), matching the stricter check already used at permanent-delete time — closing the gap where a product can be soft-deleted while still active on a purchase bill.
- Decide whether "cancelled" purchase bills should count toward a vendor's totals on the Purchases dashboard — today they do in some reports and not in others, which could confuse a business user comparing two screens.
- Re-confirm whether Factory Reset should also remove manager accounts — its own code comment describes wiping "every staff account," but the actual query only removes `role: "staff"`, leaving manager accounts (and their permission grants) behind.

Architecture & developer experience

- Update `CLAUDE.md` / `STRUCTURE.md` to reflect the manager role, section permissions, credit notes, GST filing, business branding, and factory reset — all real, shipping features these internal notes currently omit or mis-describe.
- Consider having the server independently confirm whether an invoice is "inter-state" from its place-of-supply and the business's own state, rather than trusting the client-supplied flag as-is — a small integrity improvement for GST correctness.

Business features worth considering

- A genuine "manual stock adjustment" screen — the ledger already has a reserved movement type for this, but no page currently uses it, so correcting a stock-take discrepancy today has no dedicated, audited path.
- Multi-user simultaneous notifications (e.g. "someone else is editing this invoice right now") to make the existing conflict-detection feel proactive rather than something only discovered after clicking Save.
- Scheduled/automatic reminder emails for overdue invoices, building on the email infrastructure that already exists.

AI & automation opportunities

- Automatic GST-rate/HSN suggestions when adding a new product, to reduce manual data-entry mistakes.
- Anomaly flags on the dashboard (e.g. "this invoice's total is unusually high for this customer") drawing on the same historical data already collected in the ledger and reports.

End of document. Prepared from a direct, verified reading of the Science Hub source code at `d:\nextApps\science-hub` on 2026-07-24. Every specific claim in this document traces back to an actual file and, where cited during research, a specific line number — nothing here was inferred without being explicitly flagged as such in the callout boxes above.